Tarea 1: Caracterización espectral y prototipado de filtros para reducción de ruido

CE 1110 – Análisis de Señales Mixtas

Jorge Pablo Porras Alvarado
Marcelo Matey Rodríguez
Josué Adrián Pérez Pérez

Grupo 03

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores

Profesor: Luis Chavarría Zamora

14 de octubre de 2026

Índice

1. Fundamentos, muestreo y FFT **3**
1.1. Serie de Fourier, transformada de Fourier, DFT y FFT . . . 3
1.2. Resolución frecuencial, latencia y memoria . . . 3
1.3. Fuga espectral: bin exacto frente a bin intermedio . . . 4
1.4. Simetría conjugada . . . 5
1.5. Magnitud y fase con desfase conocido . . . 5

2. Generación y caracterización de señales **5**
2.1. Evidencia por caso . . . 6
2.2. Características que permiten identificar automáticamente el ruido . . . 8
2.3. Información no recuperable cuando señal y ruido comparten frecuencias . . . 8

3. Diseño del filtro temporal **8**
3.1. Especificación . . . 9
3.2. Justificación del tipo y orden . . . 9
3.3. Coeficientes, $H(z)$ y ecuación de diferencias . . . 9
3.4. Polos, ceros, ROC y estabilidad . . . 9
3.5. Respuesta en frecuencia . . . 10
3.6. Verificación de la implementación propia . . . 10
3.7. Aplicación a los tres casos . . . 10
3.8. Cuantización de coeficientes, saturación y memoria . . . 11

4. Reducción espectral mediante FFT e IFFT **11**
4.1. Descripción del método . . . 11
4.2. Resultados . . . 13
4.3. Más ruido eliminado degrada la señal útil . . . 14
4.4. Barridos de parámetros . . . 15

5. Comparación y propuesta para el sistema embebido **15**
5.1. a) Mejor compromiso entre reducción de ruido y distorsión 16
5.2. b) Condiciones de falla 17
5.3. c) Frecuencia de muestreo, tamaño de bloque y formato numérico 17
5.4. d) Bibliotecas para el Raspberry Pi Pico W 18
5.5. e) Arquitectura de los tres nodos y comunicación 18

6. Repositorio y reproducibilidad **20**

7. Declaración de uso de inteligencia artificial **20**

1. Fundamentos, muestreo y FFT

1.1. Serie de Fourier, transformada de Fourier, DFT y FFT

Las cuatro herramientas describen una señal como suma de exponenciales complejas, pero se aplican a señales de naturaleza distinta (ver, por ejemplo, [1]).

- **Serie de Fourier**: para una señal continua y periódica de periodo T_0,

$$x(t) = \sum_{k=-\infty}^{\infty} c_k \, e^{j2\pi kt/T_0}, \qquad c_k = \frac{1}{T_0} \int_0^{T_0} x(t) \, e^{-j2\pi kt/T_0} \, dt. \tag{1}$$

  El espectro es discreto (líneas en k/T_0).

- **Transformada de Fourier**: para una señal continua no periódica,

$$X(f) = \int_{-\infty}^{\infty} x(t) \, e^{-j2\pi ft} \, dt. \tag{2}$$

  El espectro es continuo.

- **DFT**: para una secuencia finita de N muestras $x[n]$,

$$X[k] = \sum_{n=0}^{N-1} x[n] \, e^{-j2\pi kn/N}, \qquad x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] \, e^{j2\pi kn/N}. \tag{3}$$

- **FFT**: no es una transformación distinta, sino un *algoritmo* que calcula exactamente la DFT. Evaluar la definición directamente cuesta $O(N^2)$ multiplicaciones complejas; la FFT (por ejemplo, Cooley–Tukey radix-2 con N potencia de dos [2]) explota la simetría de los factores $W_N^{kn} = e^{-j2\pi kn/N}$ para reducirlo a $O(N \log_2 N)$.

La distinción central es que la DFT es la *transformación matemática* (define qué resultado se obtiene) y la FFT es su *algoritmo de cálculo* (define cómo se obtiene de forma eficiente). Ambas producen el mismo resultado salvo error numérico.

1.2. Resolución frecuencial, latencia y memoria

Con frecuencia de muestreo f_s y un bloque de N muestras, la ventana dura $T = N/f_s$ y la separación entre muestras espectrales es

$$\Delta f = \frac{f_s}{N} = \frac{1}{T}. \tag{4}$$

Mejorar la resolución (reducir Δf) exige aumentar N, lo que incrementa la latencia (hay que esperar T segundos para tener un bloque) y la memoria (el espectro complejo de N muestras en `float32` ocupa $8N$ bytes). La Tabla 1 se calculó con $f_s = 16$ kHz (`analisis_2.1.py`).

Tabla 1: Resolución, duración del bloque y memoria del espectro complejo ($f_s = 16$ kHz, `float32`).

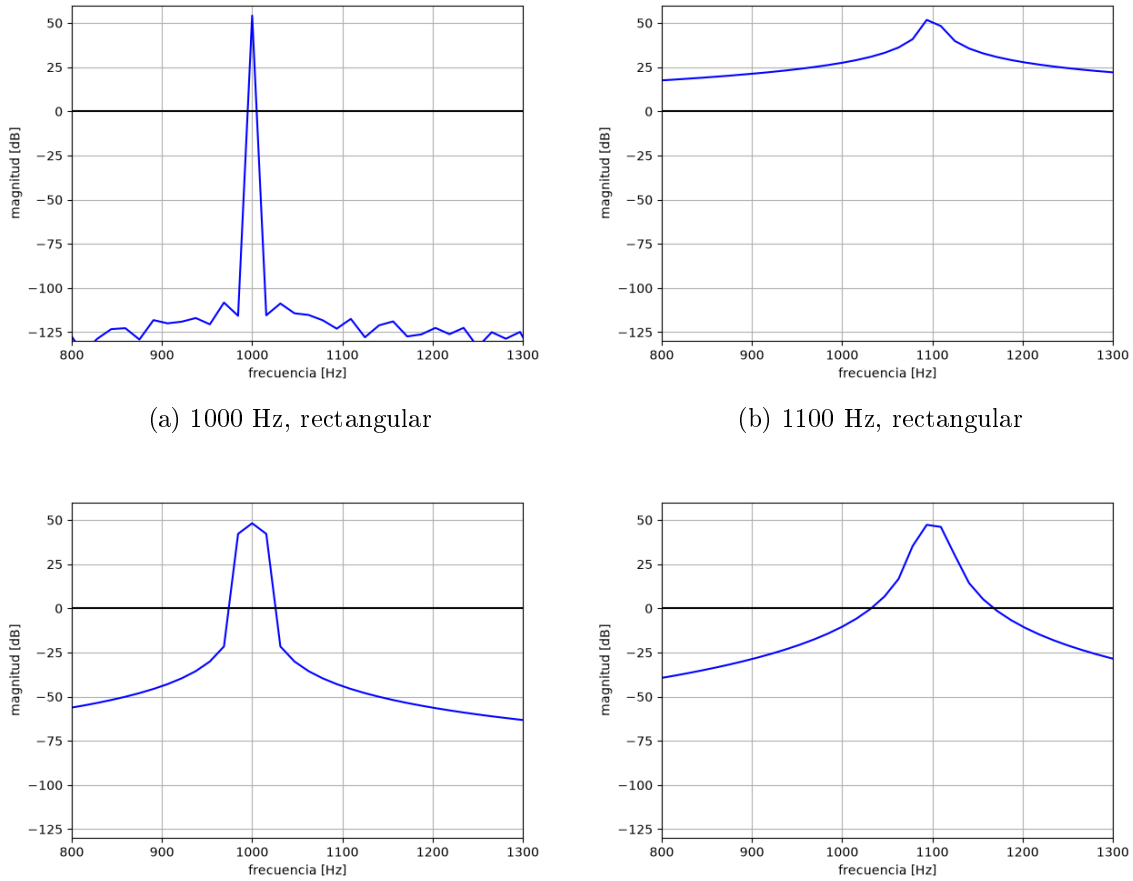
N	Δf [Hz]	$T = N/f_s$ [ms]	Memoria [KiB]
256	62.50	16	2
512	31.25	32	4
1024	15.63	64	8
4096	3.91	256	32

El compromiso es directo: con $N = 256$ la latencia es baja pero dos tonos separados menos de 62.5 Hz no se distinguen; con $N = 4096$ la resolución es excelente, pero cada bloque tarda 256 ms y consume cuatro veces más memoria que con $N = 1024$.

1.3. Fuga espectral: bin exacto frente a bin intermedio

Se generaron dos tonos con $N = 1024$ y $f_s = 16$ kHz (por lo que $\Delta f = 15{,}625$ Hz): uno de 1000 Hz, que cae exactamente en el bin $k = 64$, y otro de 1100 Hz, que cae en $k = 70{,}4$, es decir, entre dos bins. Se compararon con ventana rectangular y con ventana de Hann,

$$w[n] = 0{,}5\left(1 - \cos\frac{2\pi n}{N-1}\right). \tag{5}$$



(a) 1000 Hz, rectangular

(b) 1100 Hz, rectangular

(c) 1000 Hz, Hann

(d) 1100 Hz, Hann

Figura 1: Magnitud de la FFT para un tono en un bin exacto (1000 Hz) y entre bins (1100 Hz), con ventana rectangular y de Hann.

Interpretación.

- Cuando el tono coincide con un bin y se usa ventana rectangular, el bloque contiene un número entero de periodos y la energía se concentra en un solo bin; los demás bins son prácticamente cero (limitados por el error numérico).
- Cuando el tono cae entre bins, el bloque corta el ciclo a la mitad, lo que equivale a una discontinuidad periódica: la energía se *fuga* hacia bins vecinos y aparecen lóbulos laterales que decaen lentamente (el primero en aproximadamente -13 dB respecto al lóbulo principal).
- La ventana de Hann suaviza los extremos del bloque, por lo que los lóbulos laterales caen mucho más rápido (el primero en aproximadamente -31 dB y con decaimiento de 18 dB/octava [3]). El precio es un lóbulo principal más ancho (4 bins en lugar de 2), es decir, menor capacidad de separar tonos muy cercanos.

En este trabajo se usa Hann (en forma de raíz cuadrada en el método espectral, ver sección 4) porque la fuga de un tono intenso enmascararía componentes débiles de ruido.

1.4. Simetría conjugada

Para una señal real $x[n]$, la DFT cumple

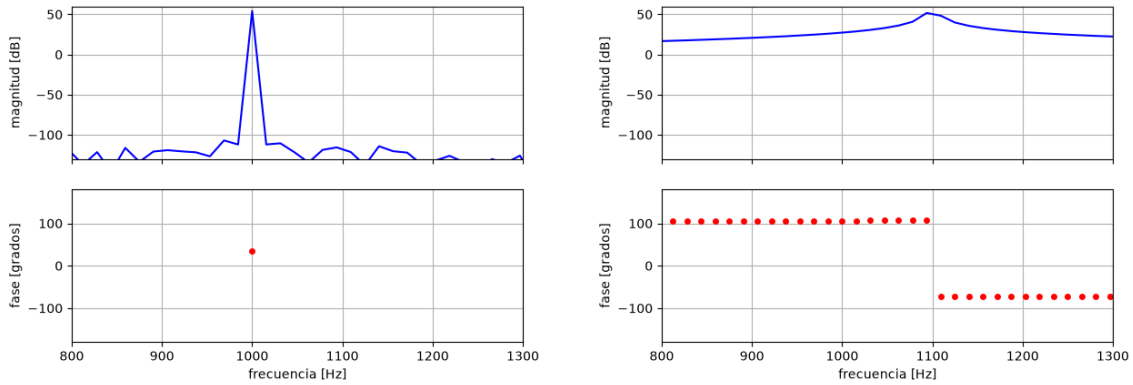
$$X[N-k] = X^*[k]. \tag{6}$$

Se verificó numéricamente con el tono de 1000 Hz: el máximo de $|X[k] - X^*[N-k]|$ sobre todos los k resultó de orden de redondeo: $3{,}8 \times 10^{-14}$.

Para comprobar por qué debe preservarse al modificar el espectro, se alteró un solo bin ($X[64] = 1 + 1j$) sin tocar su espejo. La IFFT dejó de ser real: la parte imaginaria máxima fue 0,499. Al corregir el espejo ($X[N-64] = 1 - 1j$), la parte imaginaria volvió a nivel de redondeo ($6{,}6 \times 10^{-17}$). Por lo tanto, cualquier ganancia aplicada al espectro debe ser real y simétrica en k y $N-k$; en la implementación se logra usando `rfft`/`irfft`, que solo manejan la mitad no redundante del espectro y reconstruyen el resto por simetría.

1.5. Magnitud y fase con desfase conocido

Se generó $x[n] = \cos(2\pi f n/f_s + \varphi)$ con $\varphi = 35°$. En el bin exacto (1000 Hz, $k = 64$) la fase medida fue 35,0000° (error de $\sim 10^{-12}$ grados), lo que confirma que la FFT implementada recupera la fase del tono. La fase solo es significativa donde hay energía; en el resto del espectro es ruido numérico, por lo que en las gráficas se enmascaran los bins con magnitud menor al 1 % del máximo.



(a) 1000 Hz (b) 1100 Hz

Figura 2: Magnitud y fase de la FFT para $\varphi = 35°$.

2. Generación y caracterización de señales

Los tres casos se generaron con el notebook del profesor, con semillas distintas y archivos WAV de señal limpia, ruido y señal contaminada. La Tabla 2 resume su configuración (leída de `metadatos_prueba.json` por `analisis_2.2.py`).

Tabla 2: Configuración de los casos de prueba.

	Caso 1	Caso 2	Caso 3
Tipo de ruido	tonal	blanco	banda
Semilla	2026105	2026104	2026106
f_s [Hz]	16000	16000	16000
Duración [s]	6.0	6.0	6.0
Muestras N	96000	96000	96000
Frecuencias útiles [Hz]	440, 880, 1320	440, 880, 1320	440, 880, 1320
Amplitudes útiles	1.0, 0.55, 0.3	1.0, 0.55, 0.3	1.0, 0.55, 0.3
Fases [grados]	0, 35, −50	0, 35, −50	0, 35, −50
Interferencia / banda [Hz]	1750 (tono)	todo el espectro	500–1500
SNR objetivo [dB]	5.0	5.0	3.0
SNR obtenida [dB]	5.000	5.000	3.000
Diferencia [dB]	$1{,}3 \times 10^{-14}$	8×10^{-15}	$2{,}7 \times 10^{-14}$

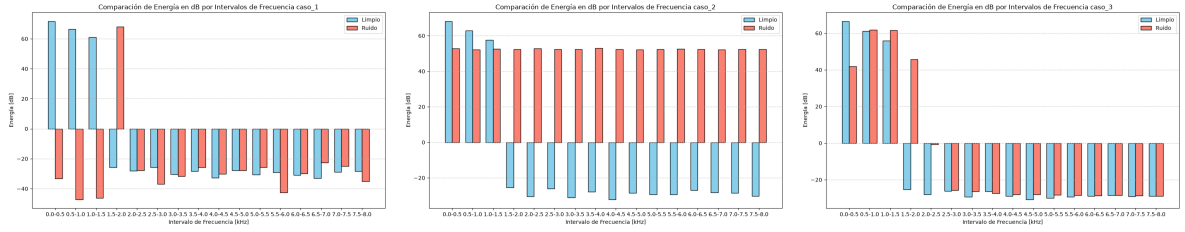
La SNR obtenida se calcula como

$$\mathrm{SNR}_{\mathrm{dB}} = 10 \log_{10} \frac{\sum_n s^2[n]}{\sum_n (x[n] - s[n])^2}, \tag{7}$$

con s la señal limpia y x la contaminada. Las SNR de la tabla provienen de `metadatos_prueba.json`; al recalcularla desde los WAV (cuantizados a `int16`) se obtiene 5.000, 5.000 y 3.000 dB, es decir, la diferencia con el objetivo es únicamente de redondeo.

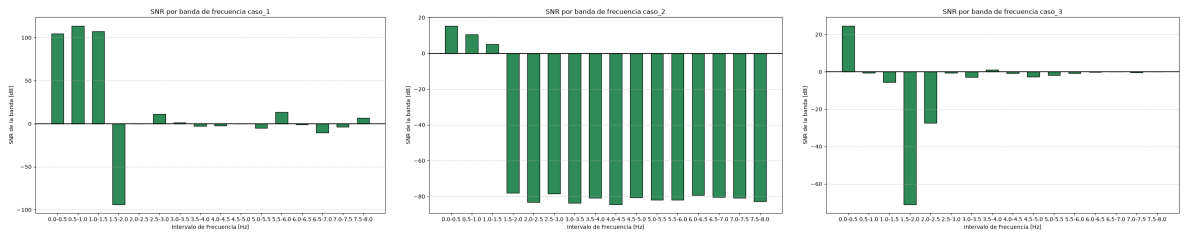
2.1. Evidencia por caso

Para cada caso se calculó el espectro (ventana de Hann, $N = 32768$ muestras desde $t = 1$ s), la fase, la energía por bandas de 500 Hz y la SNR por banda.



(a) Caso 1 (b) Caso 2 (c) Caso 3

Figura 3: Energía por banda de frecuencia: señal limpia frente a ruido.



(a) Caso 1 (b) Caso 2 (c) Caso 3

Figura 4: SNR por banda de frecuencia.

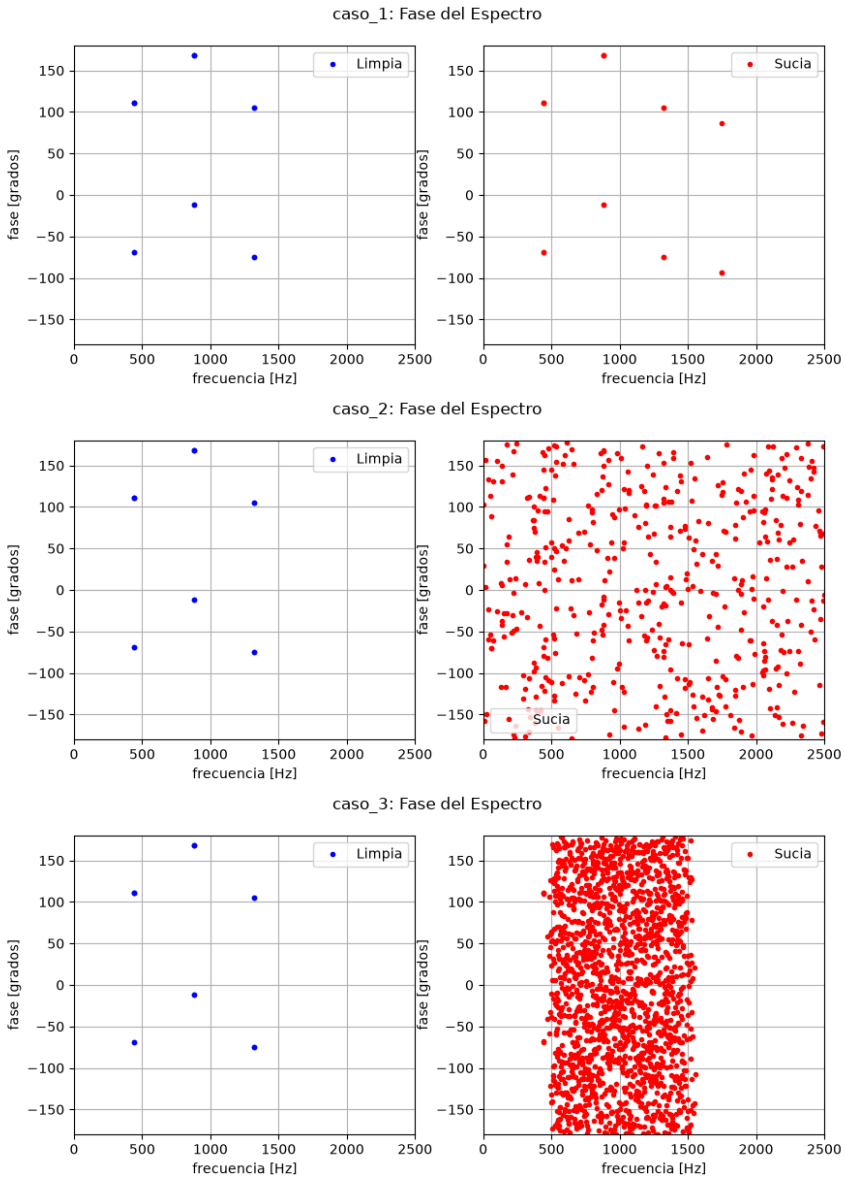


Figura 5: Fase del espectro de la señal limpia (izquierda) y contaminada (derecha) para los casos 1, 2 y 3.

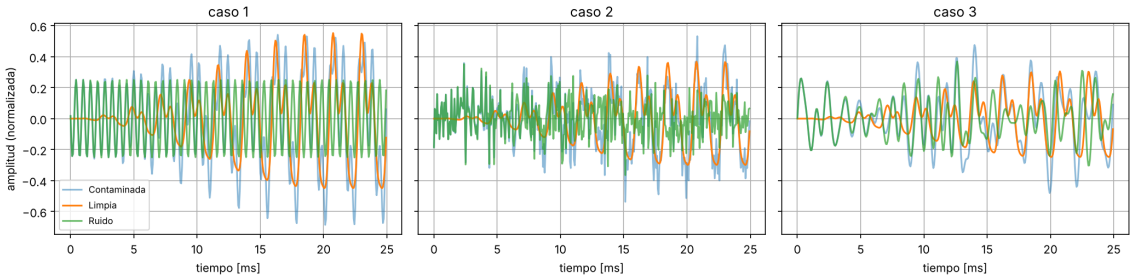


Figura 6: Señal limpia, ruido y señal contaminada en el tiempo (primeros 25 ms) para los tres casos.

Lectura de las gráficas. En el caso 1 toda la energía del ruido está en una sola línea espectral a 1750 Hz: la SNR por banda es de más de 100 dB en las bandas que solo tienen señal y de $-93{,}8$ dB en la banda de 1.5 a 2.0 kHz, donde solo hay interferencia (el valor extremo se debe

al piso numérico de 10^{-12} usado en el cálculo en dB). En el caso 2 el ruido ocupa todas las bandas con SNR positiva de 15.3, 10.6 y 5.1 dB en las tres primeras (la señal útil está concentrada ahí) y negativa por encima de 1.5 kHz. En el caso 3 la SNR por banda cae a $-0{,}6$ y $-5{,}6$ dB en 0.5–1.0 y 1.0–1.5 kHz, que es donde el ruido se superpone con los tonos de 880 y 1320 Hz. La fase de la señal contaminada solo coincide con la de la limpia en los bins de los tres tonos útiles cuando el ruido es débil en ellos (caso 1); en los casos 2 y 3 el ruido altera la fase de los bins vecinos.

2.2. Características que permiten identificar automáticamente el ruido

`analisis_2.2.py` calcula dos medidas sobre el espectro de potencia $P[k] = |X[k]|^2$ del ruido:

$$\text{pico/mediana} = 10\log_{10}\frac{\max_k P[k]}{\text{mediana}_k\, P[k]}, \qquad \text{planitud} = \frac{\exp\big(\overline{\ln P}\big)}{\overline{P}}. \tag{8}$$

La planitud espectral (cociente entre media geométrica y aritmética) vale 1 para un espectro perfectamente plano. Valores obtenidos:

Tabla 3: Medidas sobre el espectro del ruido de cada caso.

Caso	Pico/mediana [dB]	Planitud espectral
1 (tonal)	236.6	0.000
2 (blanco)	11.9	0.560
3 (banda)	98.1	0.000

El ruido blanco se distingue con claridad: pico/mediana bajo y planitud alta (0.56). Los casos 1 y 3 tienen planitud cercana a 0 porque hay bins sin energía de ruido, por lo que la planitud no basta para separar un tono de una banda. Lo que los distingue es la *concentración*: en el caso 1 la energía está en un solo pico (a 1750 Hz), mientras que en el caso 3 el 98 % de la energía del ruido cae en un rango contiguo de 500 a 1500 Hz. Un clasificador automático puede, por tanto, usar primero la planitud (¿es blanco?) y luego el ancho que ocupa la energía (un pico frente a una banda).

Además, la *persistencia* de un pico a lo largo de bloques consecutivos distingue una interferencia estacionaria de un tono útil transitorio, y la ubicación del pico fuera de la banda útil conocida permite clasificarlo como interferencia. Estas dos ideas son las que usa el método espectral de la sección 4.

2.3. Información no recuperable cuando señal y ruido comparten frecuencias

Si en un bin k coexisten señal y ruido, $X[k] = S[k] + V[k]$, solo se observa la suma. Una ganancia por bin escala ambos términos por igual, de modo que reducir $|V[k]|$ implica reducir $|S[k]|$ en la misma proporción. Tampoco es recuperable la fase de $S[k]$ si $V[k]$ es comparable en magnitud, porque $\angle X[k] \neq \angle S[k]$. Con un ruido aleatorio, solo se puede estimar su potencia promedio, no su realización exacta, así que el término $V[k]$ concreto de cada bloque no puede restarse.

3. Diseño del filtro temporal

Se eligió el **caso 1** (interferencia sinusoidal separada de los tonos útiles) y un filtro de rechazo de banda estrecha (*notch*) IIR de segundo orden, diseñado con `scipy.signal.iirnotch` [6] y aplicado con `scipy.signal.lfilter` para la verificación.

3.1. Especificación

- Frecuencia a rechazar: $f_0 = 1750$ Hz, equivalente a $\omega_0 = 2\pi f_0/f_s = 0{,}687$ rad/muestra.
- Factor de calidad $Q = 10$, por lo que el ancho de banda a -3 dB es $f_0/Q = 175$ Hz.
- Banda de paso: el resto del espectro, con ganancia cercana a 0 dB y sin rizado apreciable fuera de la vecindad del notch.
- Atenuación deseada en f_0: máxima posible (el cero está sobre el círculo unitario, por lo que teóricamente es infinita).

3.2. Justificación del tipo y orden

El espectro del caso 1 muestra una interferencia estrecha y aislada de los tonos útiles (el tono útil más cercano, 1320 Hz, está a 430 Hz), por lo que basta rechazar una banda angosta. Un IIR de segundo orden logra esto con solo 5 coeficientes y 4 estados; un FIR con transición igual de abrupta requeriría decenas de coeficientes. La contrapartida es que un IIR tiene fase no lineal, lo que se cuantifica en la sección 5.

3.3. Coeficientes, $H(z)$ y ecuación de diferencias

Los coeficientes obtenidos son

$$b = [\,0{,}96677,\ -1{,}49464,\ 0{,}96677\,], \tag{9}$$

$$a = [\,1,\ -1{,}49464,\ 0{,}93354\,]. \tag{10}$$

La función de transferencia es

$$H(z) = \frac{0{,}96677 - 1{,}49464\,z^{-1} + 0{,}96677\,z^{-2}}{1 - 1{,}49464\,z^{-1} + 0{,}93354\,z^{-2}}, \tag{11}$$

y la ecuación de diferencias correspondiente,

$$y[n] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] - a_1 y[n-1] - a_2 y[n-2]. \tag{12}$$

3.4. Polos, ceros, ROC y estabilidad

Los ceros están sobre el círculo unitario, en $z = e^{\pm j\omega_0}$ $(0{,}773 \pm 0{,}634j)$. Los polos están en $z = 0{,}966\,e^{\pm j\omega_0'}$, con $|p| = 0{,}9662$ y ángulo correspondiente a 1748 Hz. Como el sistema es causal, la región de convergencia es $|z| > 0{,}9662$, que incluye al círculo unitario; al estar todos los polos dentro de él, el filtro es estable (BIBO).

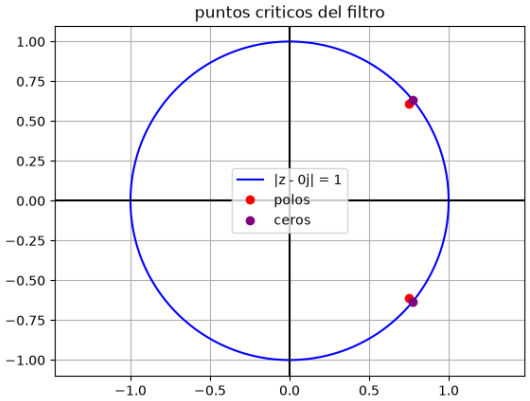


Figura 7: Diagrama de polos y ceros del notch.

3.5. Respuesta en frecuencia

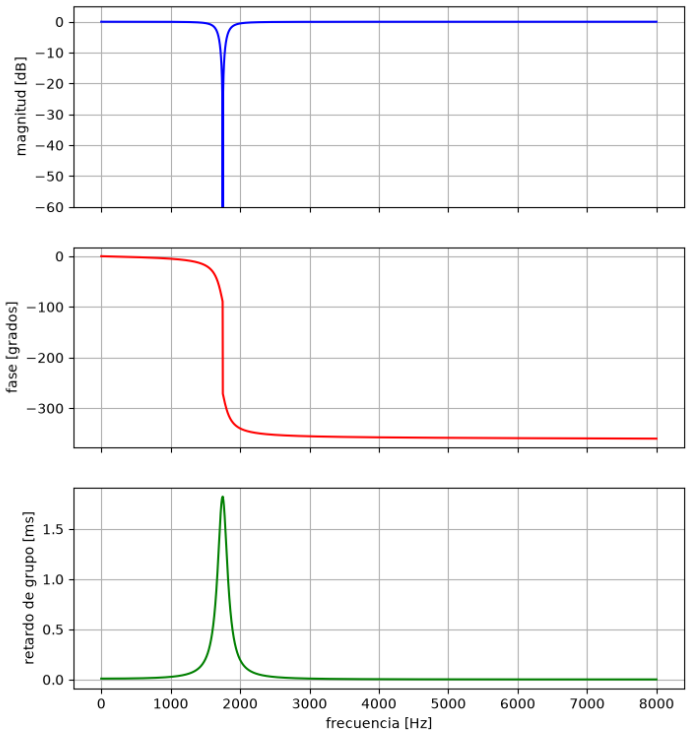


Figura 8: Magnitud, fase y retardo de grupo del notch. El retardo de grupo se omite en las cercanías de f_0, donde la fase no está definida por el cero sobre el círculo unitario.

La magnitud muestra una caída muy angosta en 1750 Hz y ganancia ≈ 0 dB lejos de ella. La fase cambia rápidamente alrededor de f_0, y el retardo de grupo alcanza sus máximos en los bordes de la muesca. Por esto un tono útil cercano sufre un desfase aunque su magnitud casi no cambie (sección 5).

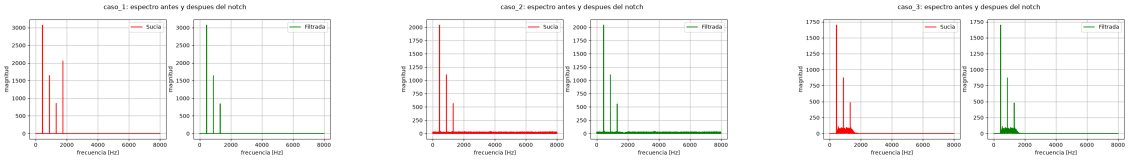
3.6. Verificación de la implementación propia

Se programó la ecuación de diferencias muestra por muestra (`ecuacion_diferencias`) y se comparó con `scipy.signal.lfilter` sobre el caso 1. El error máximo absoluto fue $6{,}7 \times 10^{-16}$, de orden del error de punto flotante, por lo que la implementación es equivalente a la de biblioteca dentro de una tolerancia de 10^{-12} (tolerancia declarada).

3.7. Aplicación a los tres casos

Tabla 4: Resultado del filtro notch sobre los tres casos (segmento desde $t = 1$ s).

Caso	$\mathrm{SNR}_{\mathrm{ent}}$ [dB]	$\mathrm{SNR}_{\mathrm{sal}}$ [dB]	ΔSNR [dB]	$\mathrm{MSE}_{\mathrm{ent}}$	$\mathrm{MSE}_{\mathrm{sal}}$
1	5.01	24.46	+19.45	$3{,}15 \times 10^{-2}$	$3{,}57 \times 10^{-4}$
2	5.02	5.11	+0.10	$1{,}39 \times 10^{-2}$	$1{,}36 \times 10^{-2}$
3	3.03	3.07	+0.04	$1{,}51 \times 10^{-2}$	$1{,}50 \times 10^{-2}$



(a) Caso 1 (b) Caso 2 (c) Caso 3

Figura 9: Espectro antes y después del notch.

El filtro elimina la interferencia del caso 1, pero casi no cambia la SNR en los casos 2 y 3, donde el ruido no está en 1750 Hz.

3.8. Cuantización de coeficientes, saturación y memoria

Se cuantizaron los coeficientes a punto fijo $Q2.m$ (redondeo al múltiplo de 2^{-m}) y a `float32`, y se midieron polos, ganancia en 1750 Hz y 1320 Hz, y SNR de salida en el caso 1.

Tabla 5: Efecto de la cuantización de coeficientes (caso 1).

Formato	máx $\lvert p\rvert$	Estable	Atenuación en 1750 Hz [dB]	$\mathrm{SNR}_{\mathrm{sal}}$ [dB]
float64	0.9662	Sí	< -200 (piso numérico)	24.46
float32	0.9662	Sí	$-117{,}7$	24.46
Q2.12	0.9662	Sí	$-57{,}0$	24.46
Q2.10	0.9662	Sí	$-39{,}8$	24.41
Q2.8	0.9662	Sí	$-19{,}6$	21.42

Con `float32` y con $Q2{,}12$ el desempeño no cambia (la SNR de salida es la misma); la degradación aparece en $Q2{,}8$, donde la atenuación en 1750 Hz cae a $-19{,}6$ dB y la SNR a 21.42 dB. En todos los formatos el filtro sigue siendo estable, pero la muesca se vuelve cada vez menos profunda: al cuantizar a_1, a_2 y b por separado, los ceros dejan de estar exactamente sobre el círculo unitario y la atenuación en f_0 ya no es infinita. La ganancia en 1320 Hz pasa de $-0{,}136$ dB (`float64`) a $-0{,}300$ dB en $Q2{,}8$. Respecto a la saturación, con coeficientes de magnitud menor a 2 y entradas normalizadas a $[-1, 1]$, un acumulador de 32 bits en $Q2{,}12$ no desborda en este filtro, aunque en $Q15$ habría que escalar a_1 (cuyo valor absoluto supera 1). La memoria del filtro es mínima: 5 coeficientes y 4 estados, es decir, 36 B en `float32`.

4. Reducción espectral mediante FFT e IFFT

4.1. Descripción del método

El método (`filtrado_espectral.py`) procesa la señal por bloques de $N = 1024$ muestras con salto $H = 512$ (50 % de solapamiento) y sigue la cadena exigida:

1. **Ventana.** Cada bloque se multiplica por $w[n] = \sqrt{\mathrm{hann}[n]}$. Aplicar esta ventana de análisis y otra igual de síntesis hace que el producto sea Hann, que con 50 % de solapamiento suma 1 y permite reconstrucción perfecta cuando la ganancia vale 1. Esto se verificó: con $G = 1$ el error máximo entre entrada y salida es de $4{,}4 \times 10^{-16}$, de orden de redondeo.

2. **FFT.** $X_m = \mathrm{rfft}(x_m w)$, que opera con tamaño potencia de dos y entrega solo la mitad no redundante del espectro.

3. **Estimación del ruido.** Se estima la potencia de ruido por bin con un filtro de mediana a lo largo de la frecuencia (ventana de 31 bins), que ignora picos aislados, y se suaviza entre

bloques con

$$\hat{N}_m[k] = \alpha\,\hat{N}_{m-1}[k] + (1-\alpha)\,\tilde{N}_m[k], \qquad \alpha = 0{,}9. \tag{13}$$

4. **Ganancia.** La ganancia total es el producto de dos términos reales, $G = G_W \cdot G_N$:

   - *Wiener* (basado en la estimación de SNR a priori de [5]; la idea de restar una estimación del ruido en el espectro viene de [4]): con $\xi[k] = \text{máx}\big(|X[k]|^2/(\beta\hat{N}[k]) - 1,\, 0\big)$,

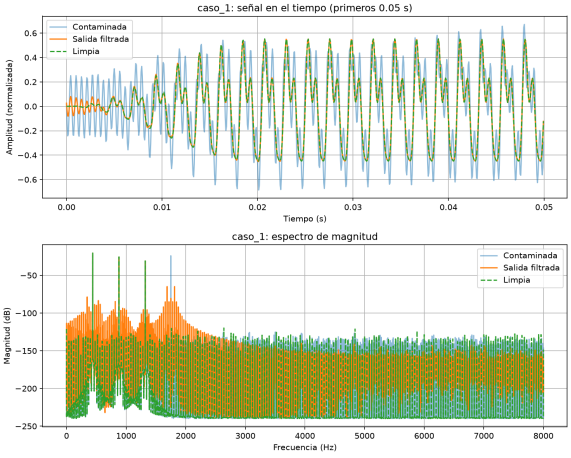
$$G_W[k] = \text{máx}\left(\frac{\xi[k]}{\xi[k]+1},\; G_{\text{mín}}\right), \qquad \beta = 1{,}5,\; G_{\text{mín}} = 0{,}1. \tag{14}$$

   - *Notch adaptativo*: se detectan picos de $|X|$ cuya prominencia supera 10 veces la magnitud típica del ruido ($\sqrt{\text{mediana}\,\hat{N}}$); si un pico queda fuera de la banda útil $[300, 1500]$ Hz, se aplica una ganancia 10^{-3} en ± 3 bins alrededor de él.

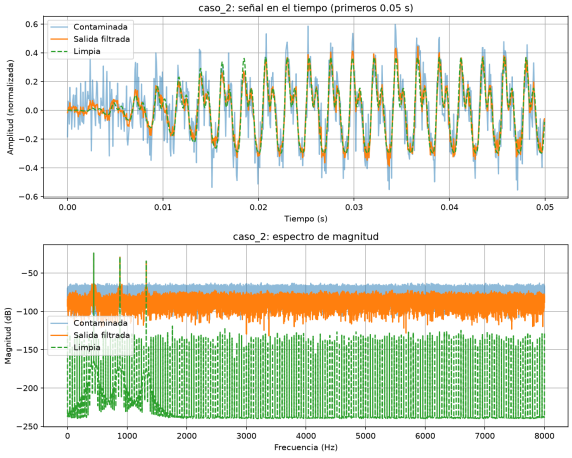
5. **IFFT y solapamiento.** $y_m = \text{irfft}(GX_m)\,w$, y las salidas se suman con solapamiento (método de solapamiento y suma, *overlap-add* [1]).

Simetría. Como G es real y se aplica sobre la mitad del espectro de `rfft`, la reconstrucción con `irfft` es siempre real (sección 1.4). **Parámetros adaptativos:** nivel de ruido por bin $\hat{N}[k]$ (se actualiza en cada bloque) y frecuencia interferente (posición del pico detectado). No se anula ningún bin de posición conocida: el pico se encuentra automáticamente en cada bloque.

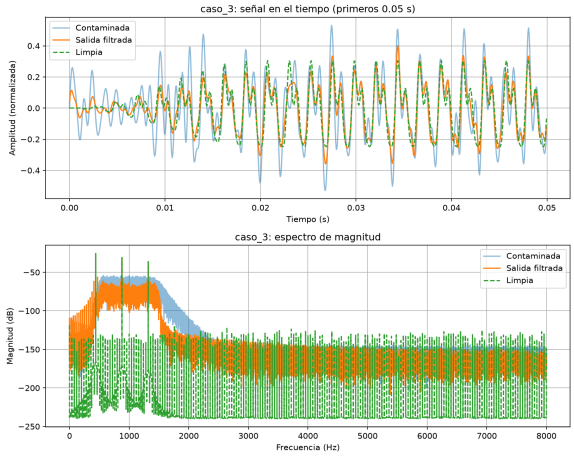
4.2. Resultados



(a) Caso 1



(b) Caso 2



(c) Caso 3

Figura 10: Señal contaminada, salida del método espectral y señal limpia (tiempo y espectro) para los tres casos.

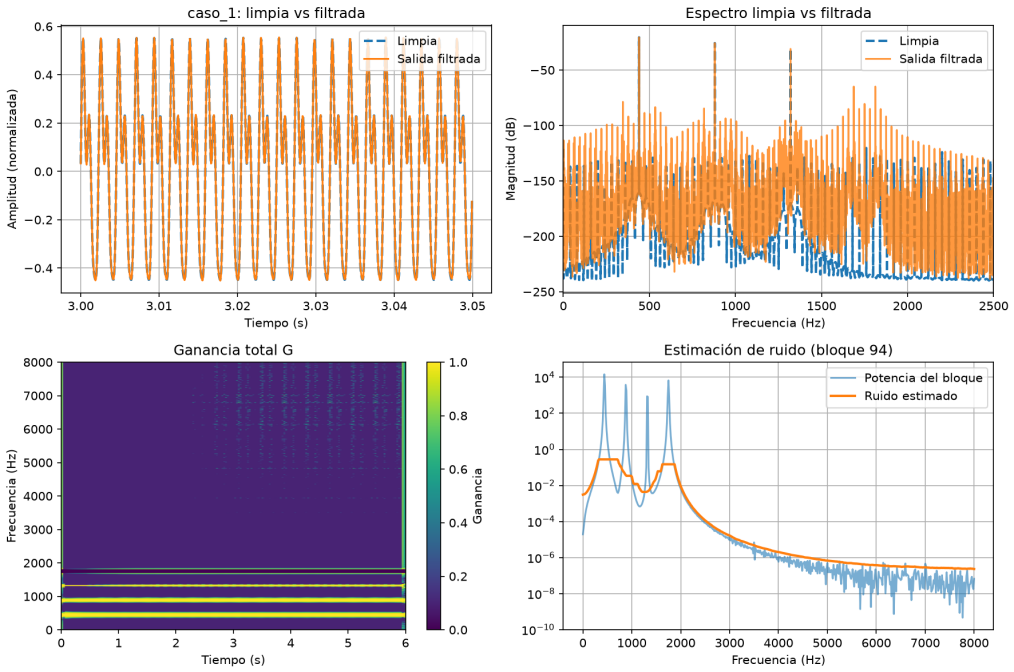


Figura 11: Diagnóstico del método en el caso 1: señal en el tiempo, espectro, mapa de ganancia $G(f,t)$ y estimación de ruido frente a la potencia del bloque.

4.3. Más ruido eliminado degrada la señal útil

Se varió la agresividad β y se midieron la energía eliminada, ΔSNR y la pérdida de amplitud en los picos de la señal útil,

$$L_u = 20\log_{10}\frac{\sum_{k\in\text{picos}}|\hat{S}[k]|}{\sum_{k\in\text{picos}}|S[k]|}. \tag{15}$$

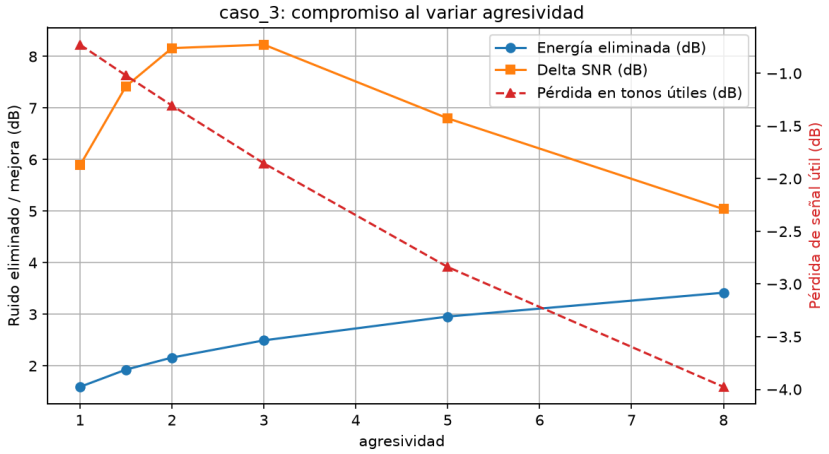


Figura 12: Compromiso al variar la agresividad β en el caso 3 (ruido solapado con la banda útil).

Tabla 6: Caso 3: efecto de la agresividad β (barrido de `filtrado_espectral.py`).

β	Energía eliminada [dB]	ΔSNR [dB]	Pérdida en tonos útiles [dB]
1	1.59	5.89	−0,73
1.5	1.93	7.42	−1,02
2	2.16	8.16	−1,31
3	2.49	8.23	−1,85
5	2.96	6.80	−2,84
8	3.42	5.04	−3,98

En el caso 3, donde el ruido se superpone con la banda útil, aumentar β elimina cada vez más energía (de 1.59 a 3.42 dB), pero el ΔSNR solo mejora hasta $\beta = 3$ (8.23 dB) y luego cae a 5.04 dB en $\beta = 8$, porque la pérdida en los tonos útiles crece de −0,73 a −3,98 dB. Más ruido eliminado significa más señal útil destruida, que es el compromiso que pide el enunciado. En los casos 1 y 2 el efecto es mucho menor: en el caso 1 la pérdida útil es de apenas −0,01 dB aun con $\beta = 8$, y en el caso 2 de −0,58 dB. Se usó $\beta = 1{,}5$ (ΔSNR de 7.42 dB con −1,02 dB de pérdida) como valor intermedio.

4.4. Barridos de parámetros

Se evaluó el efecto de `piso_ganancia`, `suavizado` y `piso_notch` sobre ΔSNR; las figuras correspondientes se generan con el script y se guardan en la carpeta de figuras con el sufijo `_espec`. Resultados en ΔSNR [dB] para los casos 1, 2 y 3:

- `piso_notch` (0,1, 0,01, 0,001): solo importa en el caso 1, donde pasa de 19.86 a 34.41 y a 35.93 dB; en los casos 2 y 3 no hay picos que atenuar y no cambia nada. Se usó 0,001.
- `piso_ganancia` (0,2 a 0,01): en el caso 1 no tiene efecto (35.93 dB), en el caso 2 sube de 8.95 a 10.30 dB y en el caso 3 de 6.96 a 7.68 dB. Se usó 0,1; bajarlo a 0,01 ganaría unos 0.5 y 0.26 dB, a costa de atenuar más cualquier bin con poca energía.
- `suavizado` (0,7, 0,9, 0,95): casi no afecta a los casos 2 y 3 (variación menor a 0.1 dB) y en el caso 1 baja de 36.63 a 35.61 dB al aumentarlo, porque la estimación de ruido reacciona más lento. Se usó 0,9.

5. Comparación y propuesta para el sistema embebido

Las métricas se calcularon sobre el segmento desde $t = 1$ s (para descartar transitorios), con

$$\mathrm{MSE} = \frac{1}{L}\sum_{n=0}^{L-1}(s[n] - \hat{s}[n])^2, \tag{16}$$

$$\mathrm{SNR}_{\mathrm{dB}} = 10\log_{10}\frac{\sum_n s^2[n]}{\sum_n (s[n] - \hat{s}[n])^2}, \tag{17}$$

$$\Delta\mathrm{SNR} = \mathrm{SNR}_{\mathrm{salida}} - \mathrm{SNR}_{\mathrm{entrada}}. \tag{18}$$

La *energía útil conservada* es el cociente entre la potencia de la salida y la de la señal limpia, sumadas en ±5 Hz alrededor de cada tono útil. La *atenuación del ruido* compara la potencia de la entrada y de la salida en los bins de ruido (excluyendo los de los tonos útiles).

Tabla 7: Mejora de SNR (ΔSNR) y energía útil conservada por método y caso (`resultados/metricas.csv`).

Caso	Temporal: ΔSNR / E. útil	Espectral: ΔSNR / E. útil
1 (tonal)	+19.4 dB / 99.6 %	+36.4 dB / 99.9 %
2 (blanco)	+0.1 dB / 100.3 %	+9.9 dB / 98.2 %
3 (banda)	+0.04 dB / 97.8 %	+7.5 dB / 87.2 %

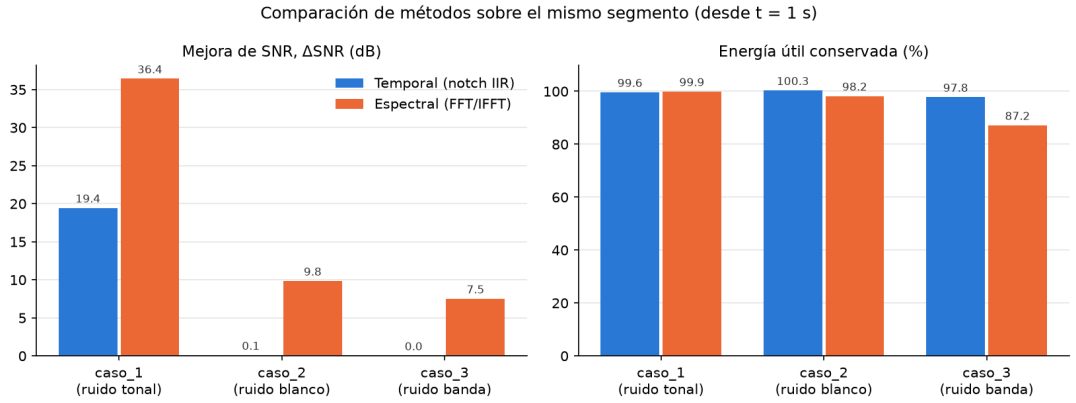


Figura 13: Comparación de ΔSNR y energía útil conservada por caso y método.

Tabla 8: Métricas completas por caso y método (`resultados/metricas.csv`).

Caso	Método	SNR [dB]	ΔSNR [dB]	MSE	E. útil [%]	Aten. [dB]	t [µs]	Mem. [B]
1	Sin filtrar	5.01	—	$3{,}15 \times 10^{-2}$	100.0	—	—	—
1	Temporal	24.46	19.45	$3{,}57 \times 10^{-4}$	99.6	195.0	8.6	36
1	Espectral	41.43	36.43	$7{,}17 \times 10^{-6}$	99.9	56.5	214.6	20 496
2	Sin filtrar	5.01	—	$1{,}39 \times 10^{-2}$	100.7	—	—	—
2	Temporal	5.11	0.10	$1{,}36 \times 10^{-2}$	100.3	0.15	8.6	36
2	Espectral	14.87	9.85	$1{,}44 \times 10^{-3}$	98.2	10.0	213.4	20 496
3	Sin filtrar	3.03	—	$1{,}51 \times 10^{-2}$	98.2	—	—	—
3	Temporal	3.07	0.04	$1{,}50 \times 10^{-2}$	97.8	0.06	8.6	36
3	Espectral	10.55	7.51	$2{,}68 \times 10^{-3}$	87.2	8.5	242.3	20 496

La memoria se cuenta en `float32` (4 B por valor); en $Q15$ serían 18 B y 10 248 B. El límite de tiempo real es de 32 000 µs por bloque.

5.1. a) Mejor compromiso entre reducción de ruido y distorsión

El compromiso consiste en reducir ruido (medido con ΔSNR y la atenuación) sin distorsionar la señal útil (energía útil conservada y MSE). El método espectral es mejor en los tres casos: logra el mayor ΔSNR (+36.4, +9.9 y +7.5 dB) y conserva al menos 98 % de la energía útil en los casos 1 y 2.

En el caso 1 el resultado requiere un matiz: el notch atenúa la interferencia más que el método espectral (más de 100 dB frente a 56.5 dB), pero logra un ΔSNR menor. La razón es que el notch introduce un desfase de aproximadamente −10° en el tono de 1320 Hz, con ganancia de solo −0,14 dB; el MSE compara muestra a muestra y penaliza ese desfase. El método espectral casi no modifica los bins fuera de los del ruido, por lo que conserva la fase de los tonos.

Con los criterios $\Delta\text{SNR} \geq 6$ dB (la potencia del error baja a una cuarta parte) y energía útil $\geq 95\,\%$ (pérdida de apenas 0.2 dB), el método espectral cumple los casos 1 y 2 pero falla el 3, y el temporal solo cumple el caso 1.

Costo computacional (en computadora, no en el microcontrolador): el temporal usa 36 B y del orden de 8 µs por bloque; el espectral, unos 20 KB y entre 213 y 242 µs por bloque. Ambos están muy por debajo del límite de 32 ms por bloque.

5.2. b) Condiciones de falla

Filtro temporal. Falla cuando el ruido no está en 1750 Hz, como se ve en los casos 2 y 3 ($\Delta\text{SNR} \approx 0$). Es un sistema LTI con un cero fijo en $e^{\pm j\omega_0}$, y solo atenúa una banda de unos 175 Hz ($Q = 10$) sin posibilidad de adaptarse. Además, aun si acierta la frecuencia, distorsiona la fase de tonos cercanos: el tono de 1320 Hz, a 430 Hz del notch, ya sufre $-10°$ y $-0{,}14$ dB.

Filtro espectral. Falla cuando el ruido comparte frecuencias con la señal útil, como en el caso 3: conserva solo 87.2 % de la energía útil y tiene el menor ΔSNR ($+7.5$ dB). El método aplica una ganancia por bin; si un bin contiene señal y ruido a la vez, no puede separarlos, y quitar el ruido implica atenuar la señal.

Limitaciones de la evidencia. Se hizo una sola corrida por caso, las señales son sintéticas y los tiempos se midieron en una computadora, por lo que no reflejan el comportamiento del microcontrolador.

5.3. c) Frecuencia de muestreo, tamaño de bloque y formato numérico

Las tres decisiones se tomaron en conjunto: f_s y el formato determinan la tasa de comunicación entre nodos, y el tamaño de bloque determina resolución, latencia y memoria.

Frecuencia de muestreo: $f_s = 16$ kHz. Para evitar aliasing, f_s debe superar el doble de la frecuencia más alta que llegue al ADC, incluido el ruido: una componente sobre $f_s/2$ se reflejaría en la banda útil y ningún filtro digital podría separarla. El modo de prueba aleatoria ubica la interferencia hasta $0{,}40 f_s = 6{,}4$ kHz y la banda de ruido hasta $0{,}46 f_s \approx 7{,}4$ kHz, ambas bajo $f_s/2 = 8$ kHz. Una frecuencia mayor no aportaría información útil y aumentaría proporcionalmente la tasa de comunicación.

Bloque: $N = 1024$ con salto de 512.

- Resolución $\Delta f = f_s/N = 15{,}6$ Hz, suficiente para separar la interferencia de los tonos útiles (el más cercano a 1750 Hz está a 430 Hz).
- Cada bloque dura $N/f_s = 64$ ms y llega un bloque nuevo cada $512/f_s = 32$ ms, que es el límite de tiempo real.
- Memoria estimada del método espectral: 5124 valores, unos 20 KB en `float32`, frente a los 264 KB de RAM del RP2040 [9] (parte de la cual ocupa el intérprete de MicroPython).

Esta configuración es la usada para la Tabla 7. En la computadora, el método espectral tardó entre 213 y 242 µs por bloque, muy por debajo de los 32 ms disponibles. Ese margen debe confirmarse en el microcontrolador, que es mucho más lento que una computadora.

Formato numérico. Se propone transmitir las muestras como `int16` y procesarlas en `float32` con `ulab`. El ADC del RP2040 es de 12 bits [9], por lo que cada muestra cabe en un `int16` sin pérdida, y el Nodo 1 resta el valor medio, dejando muestras con signo. Esto fija 2 bytes por muestra para el cálculo de la tasa de comunicación. El RP2040 no tiene unidad de punto flotante, por lo que cada operación en `float` se emula por software; el tiempo por bloque debe medirse en el Pico y no extrapolarse. Según la sección 3.8, `float32` conserva la estabilidad ($|p| = 0{,}966$) y la SNR de `float64` (24.46 dB), y $Q2{,}12$ tampoco cambia el desempeño; la degradación aparece en $Q2{,}8$ (atenuación 19.6 dB, SNR 21.4 dB). Por ello no es necesario recurrir a punto fijo para los coeficientes.

5.4. d) Bibliotecas para el Raspberry Pi Pico W

El firmware se desarrollará en MicroPython, y la biblioteca seleccionada es **ulab**, un módulo en C que implementa un subconjunto de NumPy y SciPy [8, 7].

Tabla 9: Biblioteca de procesamiento seleccionada para el Raspberry Pi Pico W.

Aspecto	ulab
Funciones a usar	`ulab.numpy.fft.fft/ifft` (método espectral, Nodos 1 y 3) y `ulab.scipy.signal.sosfilt` (notch IIR, Nodo 2)
Licencia	MIT
Precisión numérica	`float` de precisión simple (32 bits) en el RP2040
Limitaciones	La FFT solo acepta longitudes potencia de 2. El firmware oficial de MicroPython no incluye ulab, por lo que se requiere una compilación que lo integre. El cálculo en `float` se emula por software (sin FPU). La memoria está limitada al heap de MicroPython.

ulab cubre las dos operaciones necesarias, por lo que no hace falta otra biblioteca. Para el filtro temporal se usará `sosfilt` sobre el bloque completo en lugar de un ciclo muestra por muestra en Python: a 16 kHz, un ciclo interpretado no cumpliría el límite de 32 ms, mientras que `sosfilt` ejecuta el ciclo en C. Para no perder continuidad entre bloques, se pasa el parámetro opcional `zi` (estados de cada sección) de un bloque al siguiente [8]. La FFT de ulab se comparará con NumPy sobre el mismo bloque, y `sosfilt` con `scipy.signal.lfilter`, reportando el error máximo con una tolerancia declarada.

5.5. e) Arquitectura de los tres nodos y comunicación

- **Nodo 1 (adquisición y análisis):** muestrea a 16 kHz con el ADC, elimina el componente DC, aplica la ventana, calcula la FFT y estima los parámetros del ruido (frecuencia interferente y nivel).
- **Nodo 2 (filtrado temporal):** recibe muestras y parámetros, aplica el notch IIR y reproduce la salida por PWM con filtro de reconstrucción. Envía su salida al Nodo 3 para comparar.
- **Nodo 3 (filtrado espectral y evaluación):** recibe muestras y parámetros, aplica FFT/IFFT, reproduce su salida por PWM y calcula las métricas de ambos métodos (MSE, SNR antes y después, energía conservada).

Como los Nodos 2 y 3 no tienen ADC propio, la señal contaminada viaja desde el Nodo 1, y la salida del Nodo 2 viaja al Nodo 3.

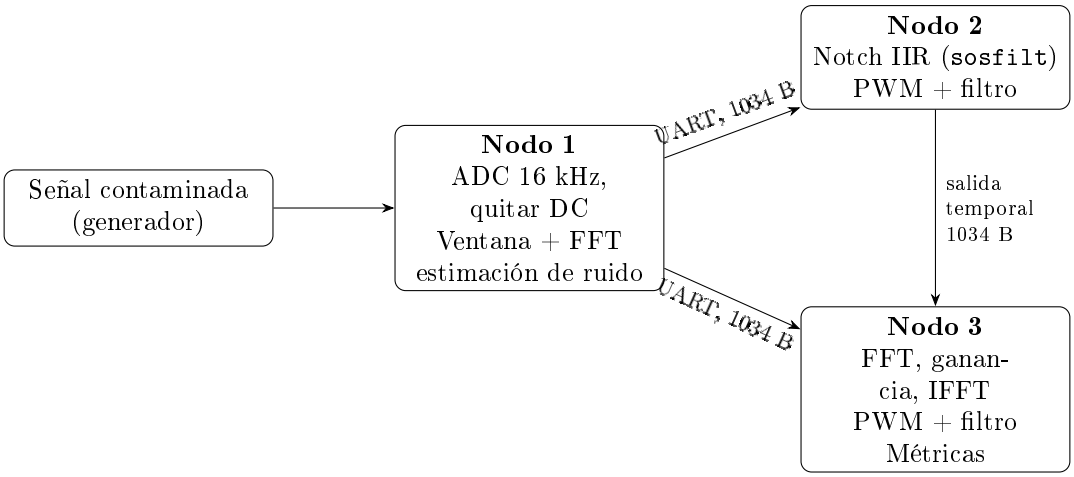


Figura 14: Diagrama de bloques de los tres nodos y datos transmitidos (UART a 460 800 baudios en todos los enlaces).

Formato de la trama. Solo se transmiten las 512 muestras nuevas de cada bloque: con 50 % de solapamiento, la otra mitad ya fue recibida en la trama anterior. La trama del Nodo 2 al Nodo 3 tiene el mismo formato, con la salida filtrada en el campo de muestras.

Tabla 10: Formato de la trama enviada por bloque.

Campo	Contenido	Bytes
Sincronía	Patrón fijo que marca el inicio del bloque	2
Número de bloque	Contador para detectar bloques perdidos	2
Parámetros	Frecuencia interferente (uint16, Hz) y nivel de ruido (uint16)	4
Muestras	512 muestras nuevas en int16	1024
CRC-16	Verificación de errores	2
Total		**1034**

Tasa mínima de comunicación.

$$\text{bloques/s} = \frac{16000}{512} = 31{,}25, \tag{19}$$

$$\text{bytes/s} = 1034 \times 31{,}25 = 32\,312 \text{ B/s por enlace}, \tag{20}$$

$$\text{tasa mínima} = 32\,312 \times 10 \text{ bits} \approx 323\,000 \text{ baudios}, \tag{21}$$

pues en UART cada byte ocupa 10 bits (8 de datos, inicio y parada).

Tabla 11: Tiempo de transmisión de una trama de 1034 B según la velocidad del UART.

UART [baudios]	Tiempo por trama [ms]	Uso del enlace	¿Cumple 32 ms?
115 200	89.8	280 %	No
460 800	22.4	70 %	Sí
921 600	11.2	35 %	Sí, con más margen

La velocidad típica de 115 200 baudios no alcanza: cada trama tardaría casi tres veces lo que dura un bloque y los búferes se desbordarían. Se propone **460 800 baudios**, que deja 30 % de margen. Si las pruebas en el Pico muestran pérdidas de bloques, el siguiente paso sería subir a 921 600 baudios, que reduce el uso del enlace a 35 %.

6. Repositorio y reproducibilidad

Repositorio: https://github.com/Palva-n/jalvarado_asm_2026_s2. Etiqueta: `tarea1_prototipo`.

Ramas.

- `generacion_analisis_senales`: generación y análisis de señales.
- `filtrado-temporal`: filtro temporal (notch IIR).
- `filtrado-espectral`: reducción de ruido mediante FFT/IFFT.
- `development`: integración del trabajo de las ramas anteriores.
- `master`: rama por defecto del repositorio.

Contribuciones.

- **Jorge Pablo Porras Alvarado:** implementación y análisis de los resultados de las secciones 2.1, 2.2 y 2.3, y redacción de la documentación.
- **Marcelo Matey Rodríguez:** implementación y análisis de la sección 2.4.
- **Josué Adrián Pérez Pérez:** sección 2.5 y notebook.

7. Declaración de uso de inteligencia artificial

Conforme al punto 6.2 del enunciado, el grupo declara el uso de una herramienta de inteligencia artificial (Claude, de Anthropic) en los siguientes aspectos:

- **Dónde se usó.** Para buscar información en la web según lo que el grupo necesitaba, para guiar implementaciones, para interpretar algunos resultados que no eran triviales y para apoyar la redacción de este informe en LaTeX.
- **Qué información se le suministró.** secciones de código desarrollado por el grupo conforme se implementaban las partes, archivos WAV y metadatos de los tres casos.
- **Cómo se verificaron los resultados.** Los valores numéricos reportados provienen de ejecutar el código del grupo sobre las señales de los tres casos. Las implementaciones se contrastaron contra funciones de biblioteca (la ecuación de diferencias contra `scipy.signal.lfilter`) y contra pruebas de consistencia (reconstrucción exacta con ganancia unitaria, simetría conjugada, SNR obtenida contra SNR objetivo). Las fuentes externas citadas se consultaron en sus páginas oficiales o en las publicaciones originales.

Referencias

[1] A. V. Oppenheim y R. W. Schafer, *Discrete-Time Signal Processing*, 3.ª ed. Upper Saddle River, NJ: Pearson, 2010.

[2] J. W. Cooley y J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series", *Mathematics of Computation*, vol. 19, n.º 90, pp. 297–301, 1965.

[3] F. J. Harris, "On the use of windows for harmonic analysis with the discrete Fourier transform", *Proceedings of the IEEE*, vol. 66, n.º 1, pp. 51–83, 1978.

[4] S. F. Boll, "Suppression of acoustic noise in speech using spectral subtraction", *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 27, n.º 2, pp. 113–120, 1979.

[5] P. Scalart y J. Vieira Filho, "Speech enhancement based on a priori signal to noise estimation", en *Proc. IEEE ICASSP*, 1996, pp. 629–632.

[6] P. Virtanen *et al.*, "SciPy 1.0: fundamental algorithms for scientific computing in Python", *Nature Methods*, vol. 17, pp. 261–272, 2020.

[7] C. R. Harris *et al.*, "Array programming with NumPy", *Nature*, vol. 585, pp. 357–362, 2020.

[8] ulab: *Documentation* (módulos `numpy.fft` y `scipy.signal`, licencia MIT). Disponible en: https://micropython-ulab.readthedocs.io/en/latest y https://github.com/v923z/micropython-ulab. Consultado el 14 de octubre de 2026.

[9] Raspberry Pi Ltd, *RP2040 Datasheet*, 2025. Disponible en: https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf. Consultado el 14 de octubre de 2026.